MachBoost: Exact Local-Context Speculative Acceleration for On-Device LLM Inference

MachBoost Authors

July 2026

Abstract

Autoregressive language model inference remains latency-bound because each generated token normally requires a serial target-model step. Speculative decoding reduces this bottleneck by proposing multiple draft tokens and verifying them against the target model, preserving the target distribution while reducing serial work. We study a practical local-first variant for developer machines: using project files, retrieved notes, prompts, and other local context as the draft source rather than training or serving a separate draft model. We introduce MachBoost, a Python package and Mac-first implementation that performs n-gram local-context drafting, target-model verification, cache-aware execution, and calibration-based gating. On an Apple M1 Max with 32 GB memory, using MLX and `mlx-community/Qwen3.5-0.8B-MLX-4bit`, MachBoost achieves a 1.38× median speedup over serial greedy decoding across 35 runs while preserving exact token equality in every run. Context-grounded workloads benefit most: policy continuation, JSON continuation, and code completion reach 1.60–1.62× median speedups, while open-ended controls slow down and are correctly identified as cases where the acceleration layer should be disabled. These results position local-context speculative acceleration as a useful systems layer for grounded on-device LLM workflows, not as a universal accelerator for arbitrary generation.

1 Introduction

Local LLM inference is increasingly attractive for privacy, offline use, cost control, and developer workflows, but it remains constrained by autoregressive decoding latency. Even when a model fits comfortably in unified memory, standard greedy decoding emits one token after another, forcing repeated model invocations. Speculative decoding attacks this problem by separating generation into drafting and verification: a cheap mechanism proposes several candidate tokens, and the target model verifies whether those tokens are exactly the same tokens it would have produced.

The central observation behind MachBoost is narrower and more practical than general speculative decoding. Many local tasks are not open-ended. They ask the model to continue a config, copy a command, complete code near existing code, quote a retrieved policy, or answer from a local document. In these settings, the next output tokens frequently occur in the prompt, retrieved context, repository, or document set. A draft model may be unnecessary: a local corpus lookup can often provide useful candidate continuations.

This paper presents MachBoost, a local-context speculative acceleration layer for verifier-capable local runtimes. MachBoost is intentionally exact: a boosted generation is acceptable only when the emitted token sequence matches the target model's serial greedy output. The system is implemented as a Python package with high-level MLX and Hugging Face adapters, a lower-level

verifier service protocol, a benchmark API, and a calibration gate that can disable acceleration for prompts where local-context speculation is not helpful.

Our contributions are:

- A practical on-device acceleration layer that uses local context as a draft source and verifier hooks in the target runtime to preserve exact greedy output.
- A cache-aware MLX verifier implementation with a cache-trajectory safety fix for non-trimmable prompt caches.
- A first-class Python API for benchmarking and calibrating whether the acceleration layer should be enabled for a workload.
- An empirical evaluation on Apple Silicon showing 1.38× median speedup across a mixed suite, 1.60–1.62× speedups on strongly context-grounded tasks, and exact output equality across all 35 measured runs.

2 Background and Related Work

Speculative decoding was introduced as a way to accelerate autoregressive transformer inference without changing outputs by drafting candidate tokens and verifying them in parallel with the target model [4]. Speculative sampling independently developed a related draft-and-verify method for sampled decoding and reported 2–2.5× speedups in a distributed setting [3]. Subsequent work explored architecture-level or self-drafting variants. Medusa adds multiple decoding heads to predict future tokens and verifies candidate trees [2].

MachBoost is closest to methods that exploit references or prompt overlap rather than an auxiliary draft model. LLMA observes that outputs in retrieval and reference-grounded scenarios often share spans with the input reference, then verifies copied spans to preserve greedy output [9]. Prompt Lookup Decoding replaces the draft model with n-gram matching over the prompt [5]; PLD+ extends that idea by using language-model artifacts such as attention or hidden states to rank candidate spans [6]. MachBoost differs in emphasis: it is a local package and systems layer for on-device runtimes, uses local file and document corpora as first-class draft sources, exposes calibration and JSON benchmark outputs, and explicitly treats low-overlap prompts as a gateable failure mode.

The implementation targets common local inference stacks. Hugging Face Transformers provides the broad model API surface for Python inference [7]. MLX is an Apple Silicon-oriented array framework and runtime used here for Mac-native evaluation [1]. Our measurements use a Qwen-family model converted for MLX; Qwen3 is the relevant public model-family report [8].

3 Problem Setting

Let M be a target autoregressive model and let x be the prompt token sequence. Greedy decoding emits

$$y_t = \arg\max_v M(v \mid x, y_{<t})$$

for each generation step t. A local-context acceleration layer receives a corpus C derived from prompt text, retrieved context, local files, or user-provided strings. The goal is to reduce wall-clock latency while returning exactly the same token sequence y that greedy decoding with M would return.

MachBoost focuses on exact greedy decoding. Sampling-compatible speculative decoding is possible in principle, but the present implementation and evaluation use greedy argmax verification because it provides a simple equality oracle: the boosted sequence must match the serial baseline token-for-token.

4 Method

MachBoost has three stages: local-context drafting, target verification, and calibration.

4.1 Local-Context Drafting

The drafter indexes token n-grams from a local corpus C. At generation time it considers the current prefix $p = x \| \hat{y}$, takes suffixes of p, and searches for matching n-grams in C. When a match is found, the tokens that follow the matched span in C become the draft candidate. Candidates are ranked by matched suffix length and draft length. This simple mechanism is cheap, deterministic, and requires no training.

4.2 Target-Model Verification

Given a prefix p and draft $d = (d_1, \ldots, d_k)$, the verifier checks how many draft tokens are equal to the target model's greedy continuation. For verifier-capable runtimes, this can be done by scoring $p \| d$ in a single target call and comparing target argmax tokens at the relevant positions. If the first a draft tokens are accepted, they are committed. If a residual target token is available at the first mismatch, it is emitted to preserve progress. Otherwise the system falls back to a normal target step.

The exactness invariant is:

$$\text{Boost}(M, x, C, T) = \text{Greedy}(M, x, T)$$

for a maximum generation length T. In the implementation, every benchmark row records an `output_match` flag by comparing boosted and baseline token IDs directly.

4.3 Cache-Aware Execution

The MLX adapter maintains target prompt-cache state across serial and verification calls. This is necessary because rebuilding a long prefix in one call is not always numerically identical to extending a cache token-by-token on local runtimes. During development we observed a real cache trajectory issue: a non-trimmable MLX cache could accept a partial draft, rebuild the cache after rejection, and later diverge from the serial cache path. MachBoost addresses this by verifying drafts on a cloned trial cache when possible. If a draft is fully accepted, the trial cache is committed. If a partial rejection occurs and the cache cannot be trimmed, MachBoost commits only the accepted tokens onto the live cache trajectory and leaves rejected tokens out. This preserves exact agreement with serial cache-fed greedy decoding.

4.4 Calibration Gate

Local-context drafting is not universally useful. Open-ended prompts often have low draft acceptance and can become slower because failed verification adds overhead. MachBoost therefore exposes a benchmark gate. For a prompt family, it measures:

- exact token match between baseline and boosted output,
- wall-clock speedup,
- accepted draft tokens divided by generated tokens,
- target-call or forward-call reduction.

The high-level API can then calibrate an accelerator:

```
from machboost import Accelerator, GatePolicy

boost = Accelerator.from_mlx(model, context_paths=["./docs", "./src"])
calibration = boost.calibrate(
    prompts,
    max_tokens=32,
    gate_policy=GatePolicy(min_speedup=1.05,
                           min_acceptance_rate=0.10),
)
text, stats = boost.generate(prompt, max_tokens=128)
```

If calibration disables the layer, `generate` falls back to the exact serial baseline path. This turns the negative controls into a product feature rather than a hidden performance hazard.

5 Implementation

MachBoost is implemented as a small Python package with no required runtime dependencies. Optional extras install backend-specific dependencies:

- `machboost[mlx]` for MLX and `mlx-lm`,
- `machboost[hf]` for PyTorch and Transformers,
- Ollama HTTP support for benchmarking and capability detection.

The core package defines a `StepService` protocol with `next_token(prefix)` and an optional `verify(prefix, candidate)` hook. Services with only `next_token` remain exact but cannot skip target work. Services with verifier hooks can reduce target calls by accepting multiple draft tokens per verification. The high-level `Accelerator` API loads MLX or Hugging Face models, reads local context from strings, files, or directories, and exposes `benchmark`, `calibrate`, and `generate`. The Ollama HTTP adapter reports that the public HTTP API lacks token IDs, logits, KV-cache snapshots, or verifier hooks; it is therefore treated as limited mode for exact acceleration.

6 Evaluation

6.1 Setup

We evaluate on a Mac with Apple M1 Max, 10 CPU cores, 32 GB unified memory, and macOS 26.5.2. The Python environment includes MLX 0.31.2 and `mlx-lm` 0.31.3. The evaluated model is `mlx-community/Qwen3.5-0.8B-MLX-4bit`. All results come from `results/mlx_evidence_suite_20260701.json`.

Metric	Value
Runs	35
Output match rate	100%
Median baseline speed	135.02 tok/s
Median boosted speed	188.13 tok/s
Median speedup	1.38×
Mean speedup	1.32×
P90 speedup	1.62×
Median accepted draft tokens	23.0
Median forward reduction	70.83%

Table 1: Overall MLX evidence-suite results. Output match compares token IDs between serial greedy baseline and boosted generation.

Fixture	Expectation	Match	Speedup	Base tok/s	Boost tok/s
Policy quote	Positive	100%	1.62×	125.84	203.78
JSON continuation	Positive	100%	1.61×	125.55	202.66
Code completion	Positive	100%	1.60×	125.13	201.84
Repo quote	Positive	100%	1.38×	136.52	188.13
RAG answer	Positive	100%	1.12×	135.02	151.57
Creative open-ended	Negative	100%	0.95×	157.63	148.91
Short answer	Negative	100%	0.90×	156.42	140.89

Table 2: Median per-fixture results over five repeats. Positive fixtures contain expected local-context overlap; negative fixtures are included to test gating.

The suite contains seven fixture families with five fresh nonce-bearing repeats each: policy continuation, structured JSON continuation, retrieval-style answering, code completion, repository command quote, creative open-ended generation, and short answer. Each run generates up to 24 tokens. The positive fixtures are designed to have real local-context overlap; the creative and short-answer fixtures are negative controls.

6.2 Overall Results

Table 1 shows that MachBoost preserves exact output equality across all runs while improving median throughput from 135.02 to 188.13 tokens per second. The median speedup is 1.38×, with p90 speedup of 1.62×.

6.3 Per-Workflow Results

Table 2 demonstrates the intended use pattern. Context-grounded continuation tasks receive substantial acceleration: policy, JSON, and code all exceed 1.60× median speedup. Repository quote achieves 1.38×. Retrieval-style answering improves only 1.12×, despite exact matching, because fewer draft tokens are accepted and verification overhead is more visible. The negative controls are exact but slower. This is expected: their generated tokens are not strongly recoverable from the local corpus, so draft attempts add overhead. A production system should gate these cases off.

6.4 Interpretation

The key result is not universal speedup. The key result is separability. MachBoost gives measurable acceleration when output tokens are recoverable from local context, and the same benchmark interface identifies low-overlap cases where the layer should be disabled. This is useful for developer tools because workloads are often repeated and classifiable: codebase-aware completion, policy extraction, config continuation, and command quoting can be calibrated once per workspace or task family.

7 Limitations

The current evidence is intentionally narrow. The main benchmark uses one small MLX model on one Apple Silicon machine with 24-token generations. Larger 3B/8B models, longer generations, sampled decoding, and broader real-world repositories remain future work. The current drafter is n-gram based and does not yet rank candidates using model artifacts, unlike PLD+. The Hugging Face path has package-level adapter support, but the reported evidence suite focuses on MLX because cache-aware local execution is the current product path. Ollama HTTP cannot expose the verifier hooks required for exact draft-token acceleration, so Ollama support is limited to benchmarking/configuration unless a native runner hook or patched runner is used.

There is also a broader methodological limitation: exact token equality is a strong correctness condition for greedy decoding, but it does not directly evaluate semantic quality under sampling. This paper therefore avoids claiming quality improvements. MachBoost is a latency layer for cases where the desired output is unchanged.

8 Future Work

Future work should evaluate 3B and 8B models, longer contexts, and realistic local repositories. The drafter can be improved with suffix automata, prompt/document segmentation, model-artifact ranking, and hybrid retrieval over multiple local corpora. The calibration gate can become adaptive online, disabling the layer after a short low-acceptance prefix and re-enabling it for structured subregions. Finally, a native Ollama or llama.cpp verifier hook would make this layer more broadly usable across popular local inference runtimes.

9 Conclusion

MachBoost demonstrates that exact local-context speculative acceleration is a practical layer for on-device LLM inference. It does not make arbitrary generation faster, and the evaluation makes that limitation visible. Instead, it accelerates the grounded workflows common in local developer environments: code, configs, policies, retrieved notes, and repeated repository text. On the measured MLX suite, it preserves exact token equality across 35 runs and achieves 1.38× median speedup overall, with 1.60–1.62× speedups on the strongest context-grounded tasks. The calibration API turns this into a usable package behavior: enable the layer when local context predicts the target output, and fall back when it does not.

References

[1] Apple Machine Learning Research. Mlx: An array framework for apple silicon. https://github.com/ml-explore/mlx, 2023. Software.

[2] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[3] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[4] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023.

[5] Apoorv Saxena. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding, 2023. GitHub repository.

[6] Shwetha Somasundaram, Anirudh Phukan, and Apoorv Saxena. Pld+: Accelerating llm inference by leveraging language model artifacts. *arXiv preprint arXiv:2412.01447*, 2024.

[7] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020.

[8] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[9] Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487*, 2023.